6 TURBO CODING

6.1 OVERVIEW

Turbo codes are binary block codes with large codewords (hundreds or thousands of bits). Turbo codes may be used to obtain even greater coding gains than those provided by concatenated coding systems. They are systematic and inherently non-transparent.

6.2 GENERAL

6.2.1 DATA RANDOMIZATION

The pseudo-randomizer defined in section 10 shall be used unless the system designer verifies that the concerns identified in the note below are resolved by other means.

NOTE – The recommended Turbo codes, by themselves, cannot guarantee sufficient bit transitions to keep receiver symbol synchronizers in lock. The pseudo-randomizer is also used to aid signal acquisition and to mitigate spectral lines in the transmitted signal.

6.2.2 FRAME VALIDATION

When Turbo codes are used with TM, AOS, or USLP Transfer Frames, the FECF specified in references [1], [2], or [6], respectively, shall be used to validate the Transfer Frame.

NOTE – While providing outstanding coding gain, Turbo codes may still leave some undetected errors in the decoded output.

6.3 SPECIFICATION

NOTE – A Turbo encoder is a combination of two simple encoders. The input is a frame of k information bits. The two component encoders generate parity symbols from two simple recursive convolutional codes, each with a small number of states. The information bits are also sent uncoded. A key feature of Turbo codes is an interleaver, which permutes bit-wise the original k information bits before input to the second encoder.

The recommended Turbo code is a systematic code that shall conform to the following specifications:

a) Code type shall be systematic parallel concatenated Turbo code.

b) Number of component codes shall be two (plus an uncoded component to make the code systematic).

c) Type of component codes shall be recursive convolutional codes.

d) Number of states of each convolutional component code shall be 16.

e) Nominal code rates shall be r = 1/2, 1/3, 1/4, or 1/6 (selectable).

NOTE – Because of 'trellis termination' symbols (see 6.3j)), the true code rates (defined as the ratios of the information block lengths to the codeblock lengths in table 6-2) are slightly smaller than the nominal code rates. In this Recommended Standard, the term 'code rate' always refers to the nominal code rates, r = 1/2, 1/3, 1/4, or 1/6.

f) The specified information block lengths k shall be those specified in table 6-1. The corresponding codeblock lengths in bits, $n=(k+4)/r$, for the specified code rates shall be those specified in table 6-2.

NOTE – Information block lengths are chosen for compatibility with the corresponding Reed-Solomon interleaving depths, also shown in table 6-1.

Table 6-1: Specified Information Block Lengths

Information block length *k*, bits	Corresponding Reed-Solomon interleaving depth *I*	Notes
1784 (=223 × 1 octets)	1	For very low data rates or low latency
3568 (=223 × 2 octets)	2	
7136 (=223 × 4 octets)	4	
8920 (=223 × 5 octets)	5	For highest coding gain

Table 6-2: Codeword Lengths for Supported Code Rates (Measured in Bits)

Information block length *k*	Codeword length *n*			
	rate 1/2	rate 1/3	rate 1/4	rate 1/6
1784	3576	5364	7152	10728
3568	7144	10716	14288	21432
7136	14280	21420	28560	42840
8920	17848	26772	35696	53544

g) Turbo code permutation for each specified block length *k* shall conform to a particular reordering of the integers 1, 2, . . ., *k* as generated by the following algorithm.

NOTE – The interleaver is a fundamental component of the Turbo encoding and decoding process. The interleaver for Turbo codes is a fixed bit-by-bit permutation of the entire block of data. Unlike the symbol-by-symbol rectangular interleaver used with Reed-Solomon codes, the Turbo code permutation scrambles individual bits and resembles a randomly selected permutation in its lack of apparent orderliness.

1) *k* shall be expressed as $k=k_1k_2$; the parameters k_1 and k_2 for the specified block sizes shall be selected from table 6-3.

Table 6-3: Parameters k_1 and k_2 for Specified Information Block Lengths

Information block length	k_1	k_2
1784	8	223
3568	8	223×2
7136	8	223×4
8920	8	223×5

2) The following operations shall be performed for s=1 to s=k to obtain permutation numbers $\pi(s)$:

$p_1 = 31$; $p_2 = 37$; $p_3 = 43$; $p_4 = 47$; $p_5 = 53$; $p_6 = 59$; $p_7 = 61$; $p_8 = 67$

NOTE – In the equation below, $\lfloor x \rfloor$ denotes the largest integer less than or equal to x, and p_q denotes one of the following eight prime integers:

$$
\begin{aligned}
m &= (s-1) \bmod 2 \\
i &= \left\lfloor \frac{s-1}{2\,k_2} \right\rfloor \\
j &= \left\lfloor \frac{s-1}{2} \right\rfloor - i\,k_2 \\
t &= (19i+1) \bmod \frac{k_1}{2} \\
q &= t \bmod 8 + 1 \\
c &= (p_q j + 21m) \bmod k_2 \\
\pi(s) &= 2\left(t + c\,\frac{k_1}{2} + 1\right) - m
\end{aligned}
$$

The permutation numbers shall be interpreted such that the sth bit read out on line ‘in b’ in figure 6-2 is the $\pi(s)$th bit of the input information block, as shown in figure 6-1.

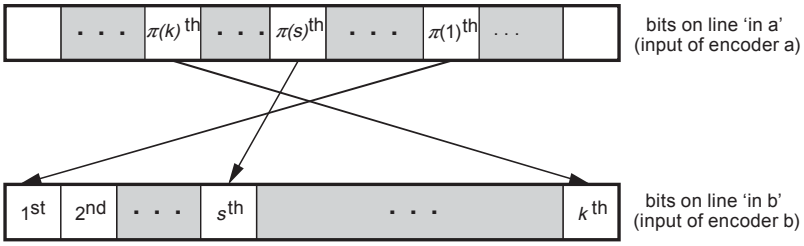


Figure 6-1: Interpretation of Permutation

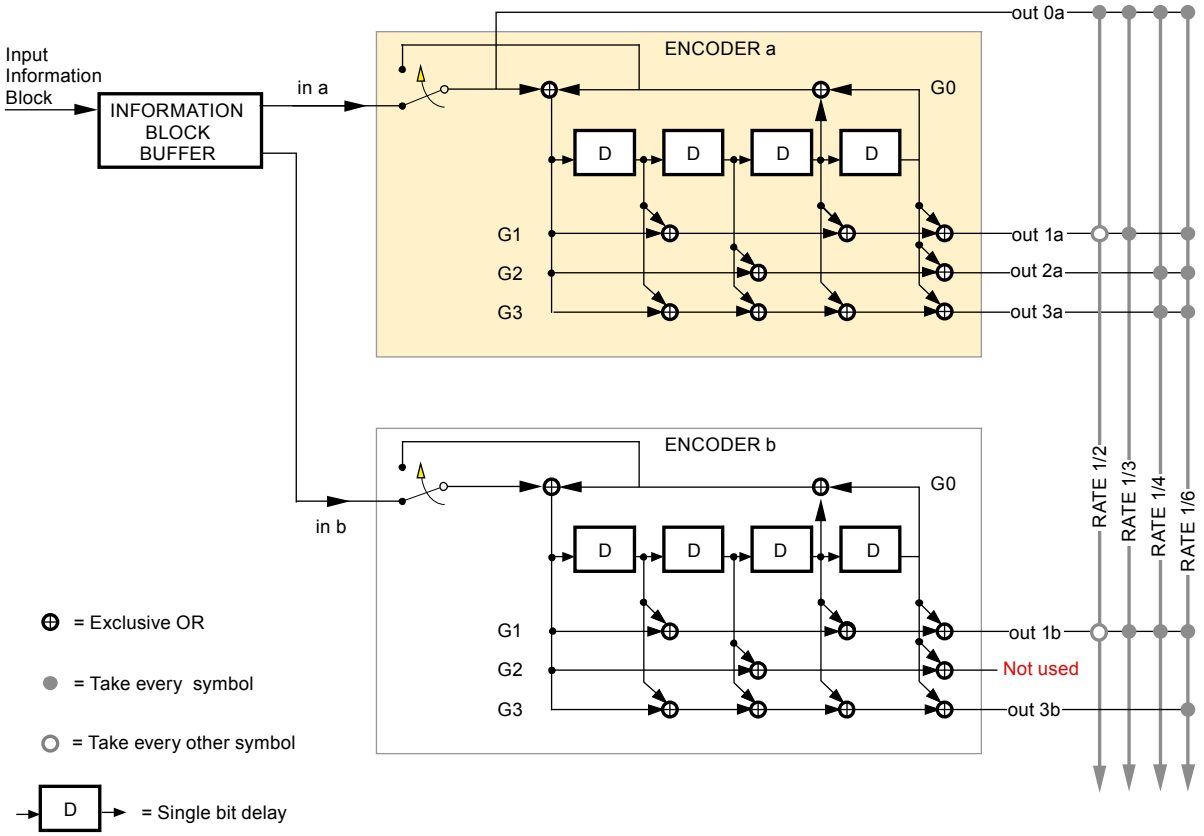


Figure 6-2: Turbo Encoder Block Diagram

h) Backward and forward connection vectors (see figure 6-2) shall be as follows:

1) Backward connection vector for both component codes and all code rates shall be G0 = 10011.

2) Forward connection vector for both component codes and rates 1/2 and 1/3 shall be G1 = 11011.

i) Puncturing of every other symbol from each component code shall be done for rate 1/2.

ii) No puncturing shall be done for rate 1/3.

3) Forward connection vectors for rate 1/4 shall be G2 = 10101, G3 = 11111 (1st component code); G1 = 11011 (2nd component code). No puncturing shall be done for rate 1/4.

4) Forward connection vectors for rate 1/6 shall be G1 = 11011, G2 = 10101, G3 = 11111 (1st component code); G1 = 11011, G3 = 11111 (2nd component code). No puncturing shall done for rate 1/6.

i) Turbo encoder operation:

1) Each input frame of *k* information bits shall be held in a frame buffer, and the bits in the buffer shall be read out in two different orders for the two component encoders.

2) The first component encoder (a in figure 6-2) shall operate on the bits in unpermuted order ('in a'), while the second component encoder (b figure 6-2) shall receive the same bits permuted by the interleaver ('in b').

NOTES

1 The read-out addressing for 'in a' is a simple counter, while the addressing for 'in b' is specified by the Turbo code permutation described in 6.3g).

2 The recommended encoder block diagram is shown in figure 6-2.

3 The component encoders are recursive convolutional encoders realized by feedback shift registers as shown in figure 6-2. The circuits shown in this figure implement the backward connection vector, G0, and the forward connection vectors, G1, G2, G3, specified in 6.3h). A key difference between these convolutional component encoders and the standalone convolutional encoder recommended in section 3 is their recursiveness. In the figure this is indicated by the signal (corresponding to the backward connection vector G0) fed back into the leftmost adder of each component encoder.

j) Turbo codeword specification:

1) Both component encoders (see figure 6-2) shall be initialized with '0's in all registers, and both shall run for a total of k+4 bit times, producing an output codeword of (k+4)/r encoded symbols, where r is the nominal code rate.

2) For the first k bit times, the input switches shall be in the lower position (as indicated in figure 6-2) to receive input data.

3) For the final four bit times, these switches shall move to the upper position to receive feedback from the shift registers.

encoder is not advised because it introduces considerable loss of performance. It also would require differential detection, which is more complex with soft symbols. This implies that phase ambiguities have to be detected and resolved before decoding.

3 A diagram of a Turbo codeword with ASM is shown in figure 6-4. The length of the Turbo codeword is inversely proportional to the nominal code rate *r*.

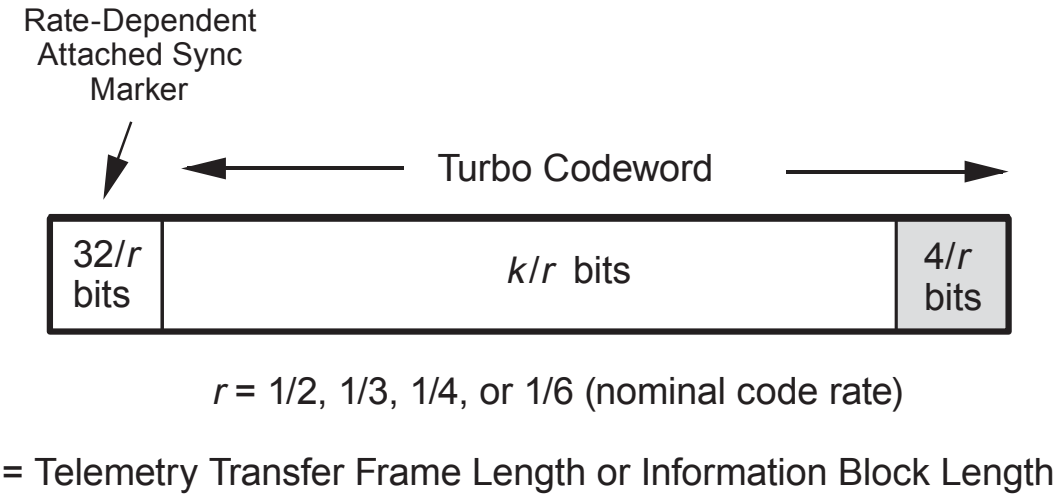


Figure 6-4: Turbo Codeword with Attached Sync Marker